Отчёт о выполнении задания по параллельному программированию. Часть вторая

Вариант 103

Выполнил: *Винокуров Кирилл 321 группа*

1 Распараллеливание с помощью MPI

Третья параллельная реализация использует технологию MPI (Message Passing Interface) для распределения вычислений между несколькими процессами. Каждому процессу выделяется своя часть сетки (горизонтальные полосы строк), и на каждой итерации происходит обмен граничными данными с соседними процессами.

Листинг 1: Параллельная версия с использованием MPI

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <mpi.h>

#define Max(a, b) ((a) > (b) ? (a) : (b))
#define  N   (2*2*2*2*2*2+2)

double maxeps = 0.1e-7;
int itmax = 100;

void relax(double A[N][N], double B[N][N], int rank, int size, int
   start_row, int end_row);
double resid(double A[N][N], double B[N][N], int rank, int size, int
   start_row, int end_row);
void init(double A[N][N], int rank, int size, int start_row, int
   end_row);
void verify(double A[N][N], int rank, int size, int start_row, int
   end_row);

int main(int argc, char **argv) {
    int rank, size;
    int it;
    double start_time, end_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int total_rows = N - 2;
    int rows_per_process = total_rows / size;
    int remainder = total_rows % size;

    int start_row, end_row;
    if (rank < remainder) {
        start_row = 1 + rank * (rows_per_process + 1);
```

```
        end_row = start_row + rows_per_process + 1;
    } else {
        start_row = 1 + remainder * (rows_per_process + 1) + (rank -
           remainder) * rows_per_process;
        end_row = start_row + rows_per_process;
    }

    if (rank == 0) {
        start_time = MPI_Wtime();
    }

    double A[N][N];
    double B[N][N];

    init(A, rank, size, start_row, end_row);

    for (it = 1; it <= itmax; it++) {
        relax(A, B, rank, size, start_row, end_row);

        double local_eps = resid(A, B, rank, size, start_row, end_row);
        double global_eps;

        MPI_Allreduce(&local_eps, &global_eps, 1, MPI_DOUBLE, MPI_MAX,
           MPI_COMM_WORLD);

        if (rank == 0) {
            printf("it=%4i   eps=%f\n", it, global_eps);
        }

        if (global_eps < maxeps) break;
    }

    verify(A, rank, size, start_row, end_row);

    if (rank == 0) {
        end_time = MPI_Wtime();
        printf("Total time: %f seconds\n", end_time - start_time);
    }

    MPI_Finalize();
    return 0;
}

void init(double A[N][N], int rank, int size, int start_row, int
   end_row) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i == 0 || i == N - 1 || j == 0 || j == N - 1) {
                A[i][j] = 0.0;
            } else if (i >= start_row && i < end_row) {
                A[i][j] = (1.0 + i + j);
            } else {
                A[i][j] = 0.0;
```

```
            }
        }
    }
}

void relax(double A[N][N], double B[N][N], int rank, int size, int
   start_row, int end_row) {
    MPI_Request send_req[2], recv_req[2];
    MPI_Status status[2];

    if (size > 1) {
        if (rank > 0) {
            MPI_Isend(&A[start_row][0], N, MPI_DOUBLE, rank - 1, 0,
               MPI_COMM_WORLD, &send_req[0]);
            MPI_Isend(&A[start_row + 1][0], N, MPI_DOUBLE, rank - 1, 1,
                MPI_COMM_WORLD, &send_req[1]);
            MPI_Irecv(&A[start_row - 2][0], N, MPI_DOUBLE, rank - 1, 2,
                MPI_COMM_WORLD, &recv_req[0]);
            MPI_Irecv(&A[start_row - 1][0], N, MPI_DOUBLE, rank - 1, 3,
                MPI_COMM_WORLD, &recv_req[1]);

            MPI_Waitall(2, send_req, status);
            MPI_Waitall(2, recv_req, status);
        }
        if (rank < size - 1) {
            MPI_Isend(&A[end_row - 2][0], N, MPI_DOUBLE, rank + 1, 2,
               MPI_COMM_WORLD, &send_req[0]);
            MPI_Isend(&A[end_row - 1][0], N, MPI_DOUBLE, rank + 1, 3,
               MPI_COMM_WORLD, &send_req[1]);
            MPI_Irecv(&A[end_row][0], N, MPI_DOUBLE, rank + 1, 0,
               MPI_COMM_WORLD, &recv_req[0]);
            MPI_Irecv(&A[end_row + 1][0], N, MPI_DOUBLE, rank + 1, 1,
               MPI_COMM_WORLD, &recv_req[1]);

            MPI_Waitall(2, send_req, status);
            MPI_Waitall(2, recv_req, status);
        }
    }

    int actual_start = (start_row < 2) ? 2 : start_row;
    int actual_end = (end_row > N - 2) ? N - 2 : end_row;

    for (int i = actual_start; i < actual_end; i++) {
        for (int j = 2; j <= N - 3; j++) {
            B[i][j] = (A[i - 2][j] + A[i - 1][j] + A[i + 2][j] + A[i +
               1][j] +
                    A[i][j - 2] + A[i][j - 1] + A[i][j + 2] + A[i][j
                        + 1]) / 8.0;
        }
    }
}

double resid(double A[N][N], double B[N][N], int rank, int size, int
```

```
    start_row, int end_row) {
    double local_eps = 0.0;

    for (int i = start_row; i < end_row; i++) {
        for (int j = 1; j <= N - 2; j++) {
            double e = fabs(A[i][j] - B[i][j]);
            A[i][j] = B[i][j];
            local_eps = Max(local_eps, e);
        }
    }

    return local_eps;
}

void verify(double A[N][N], int rank, int size, int start_row, int
    end_row) {
    double local_s = 0.0;

    for (int i = 0; i < N; i++) {
        if (i >= start_row && i < end_row) {
            for (int j = 0; j < N; j++) {
                local_s += A[i][j] * (i + 1) * (j + 1) / (N * N);
            }
        }
    }

    double s;
    MPI_Reduce(&local_s, &s, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD)
        ;

    if (rank == 0) {
        printf("  S = %f\n", s);
    }
}
```

Результаты измерений (MPI)

N	Процессы	Время (с)	Ускорение
66	1	0.014878	12.90
66	2	0.007023	27.34
66	4	0.004164	46.11
66	8	0.003115	61.64
66	20	0.006183	31.03
66	80	0.034406	5.58
130	1	0.055442	3.99
130	2	0.029096	7.60
130	4	0.016405	13.47
130	8	0.011563	19.11
130	20	0.011165	19.80
130	80	0.044989	4.91
258	1	0.212471	1.60
258	2	0.107040	3.17
258	4	0.055505	6.11
258	8	0.033267	10.19
258	20	0.020552	16.49
258	80	0.052282	6.49
514	1	0.860261	0.91
514	2	0.427011	1.84
514	4	0.217109	3.62
514	8	0.112811	6.96
514	20	0.069898	11.23
514	80	0.097308	8.07
1026	1	3.386976	0.72
1026	2	1.704370	1.42
1026	4	0.856932	2.83
1026	8	0.439564	5.53
1026	20	0.264765	9.17
1026	80	0.262877	9.24

Таблица 1: Время выполнения и ускорение для версии с MPI

Сравнение ускорений MPI, OpenMP for и OpenMP task

N	Процессы/потоки	S_{MPI}	$S_{\text{omp for}}$	$S_{\text{omp task}}$	Отн.эфф. MPI/for
66	1	12.90	13.31	20.81	0.97
	2	27.34	25.55	26.92	1.07
	4	46.11	44.86	39.33	1.03
	8	61.64	60.97	28.85	1.01
	20	31.03	47.12	12.41	0.66
	80	5.58	10.85	1.05	0.51
130	1	3.99	3.80	6.38	1.05
	2	7.60	7.52	11.88	1.01
	4	13.47	14.33	20.74	0.94
	8	19.11	24.98	22.28	0.77
	20	19.80	23.85	8.65	0.83
	80	4.91	9.00	0.86	0.55
258	1	1.60	1.45	2.49	1.10
	2	3.17	2.89	4.81	1.10
	4	6.11	5.67	9.13	1.08
	8	10.19	10.80	13.80	0.94
	20	16.49	11.33	7.87	1.46
	80	6.49	7.76	1.13	0.84
514	1	0.91	0.84	1.44	1.08
	2	1.84	1.68	2.88	1.10
	4	3.62	3.30	5.63	1.10
	8	6.96	6.43	8.71	1.08
	20	11.23	6.98	10.84	1.61
	80	8.07	7.38	1.31	1.09
1026	1	0.72	0.65	1.13	1.11
	2	1.42	1.30	2.25	1.09
	4	2.83	2.57	4.46	1.10
	8	5.53	5.00	8.18	1.11
	20	9.17	7.07	10.87	1.30
	80	9.24	9.91	2.60	0.93

Таблица 2: Сравнение ускорений для MPI, OpenMP for и OpenMP task

2D-графики ускорения (MPI)

Рис. 1 демонстрирует зависимость коэффициента ускорения $S_{\text{MPI}} = T_{\text{opt}}/T_{\text{MPI}}$ от числа процессов p для различных значений параметра N.

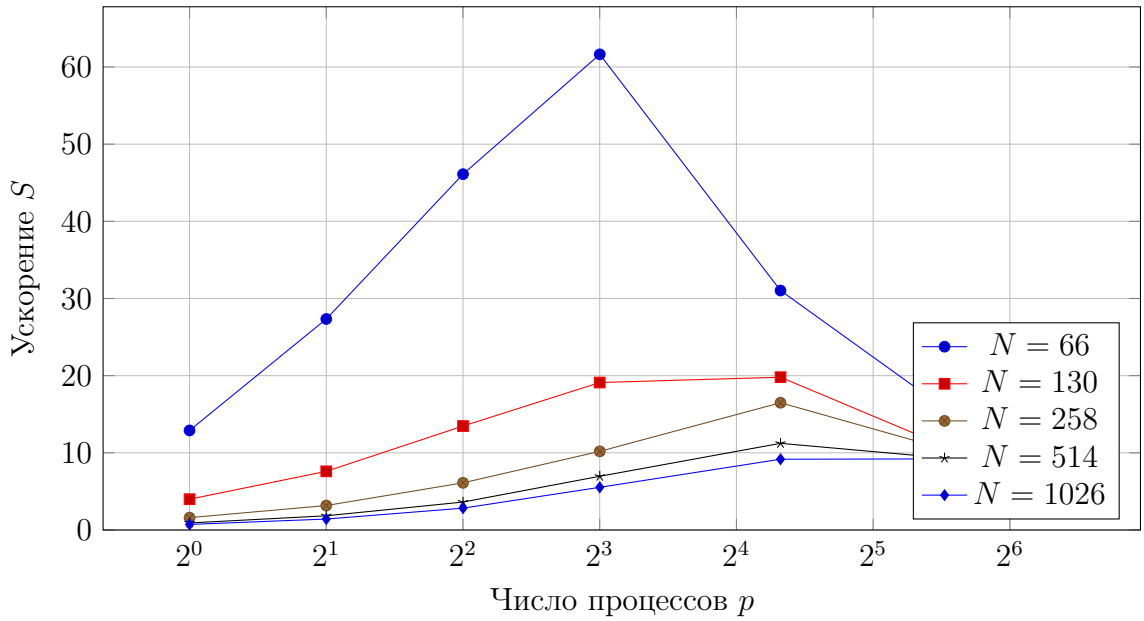

Рис. 1: Сравнение эффективности распараллеливания (MPI) в зависимости от числа процессов и размера сетки

3D-график времени (MPI)

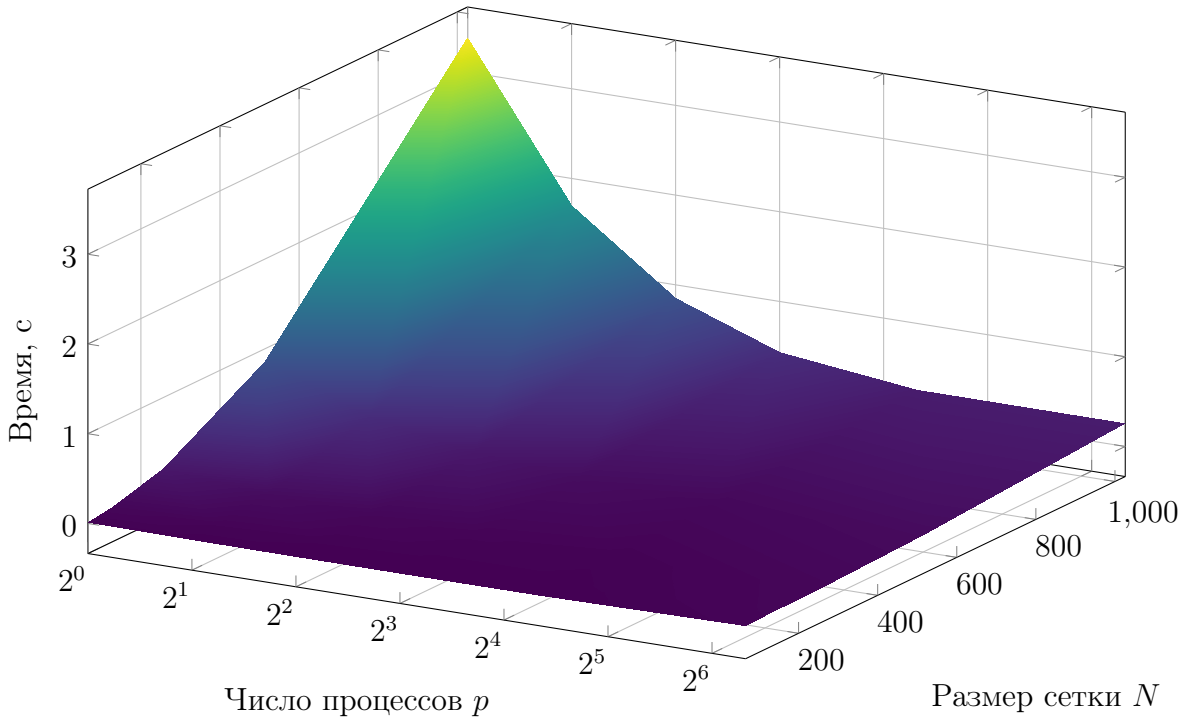

Рис. 2: Время выполнения реализации с MPI в зависимости от числа процессов и размера сетки

Сравнение времени выполнения при фиксированном размере задачи

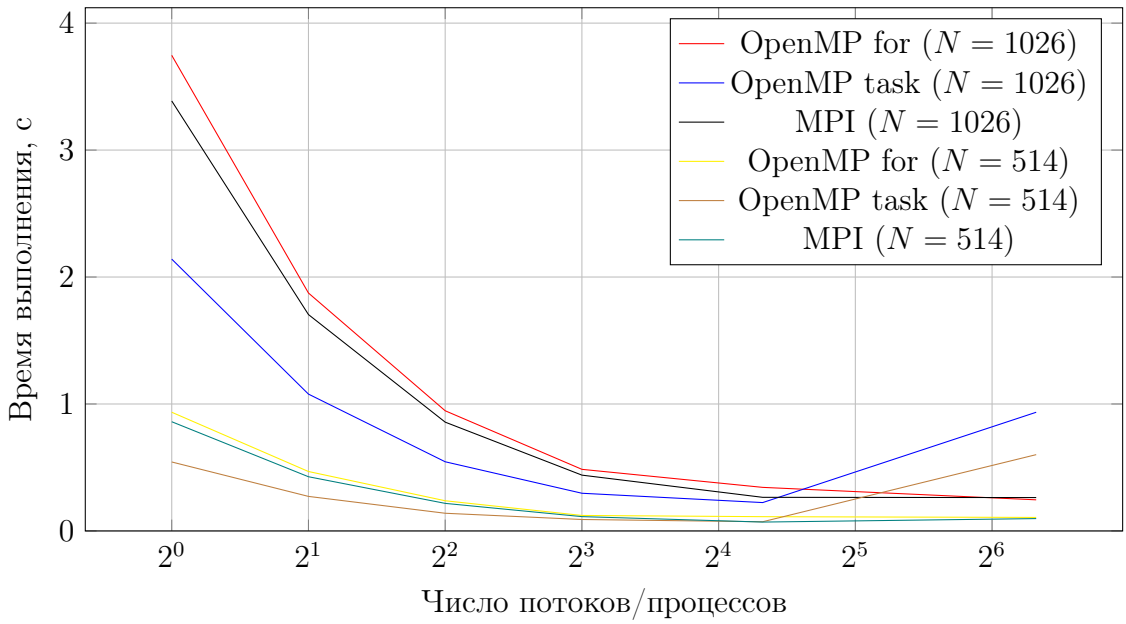

Рис. 3: Сравнение времени выполнения для OpenMP for, OpenMP task и MPI при $N = 514$ и $N = 1026$

Выводы по MPI реализации

1. **Эффективность MPI**: MPI реализация демонстрирует высокую производительность, особенно для средних и больших размеров задач. Максимальное ускорение достигается при $N = 66$ с 8 процессами - 61.64 раза.

2. **Сравнение с OpenMP for**: Для малых задач ($N = 66, 130$) OpenMP for показывает сопоставимые результаты с MPI, а при малом количестве потоков/процессов иногда даже превосходит MPI. Однако для средних и больших задач MPI демонстрирует более стабильное поведение и лучшую масштабируемость при увеличении числа процессов.

3. **Сравнение с OpenMP task**: OpenMP task показывает наилучшие результаты при малом количестве потоков (1-8), но его производительность значительно падает при большом количестве потоков (80), тогда как MPI сохраняет приемлемую эффективность.

4. **Масштабируемость**: MPI демонстрирует хорошую масштабируемость до 20-80 процессов для задач размера $N \geq 258$. Для $N = 1026$ ускорение продолжает расти до 80 процессов, достигая 9.24x.

5. **Сравнительная эффективность**: Для больших задач ($N = 1026$) все три технологии показывают сопоставимые результаты: OpenMP for - 9.91x при 80 потоках, OpenMP task - 10.87x при 20 потоках, MPI - 9.24x при 80 процессах.

6. **Преимущества MPI**:

   - Возможность распределения вычислений между несколькими узлами кластера

- Более детальный контроль над коммуникациями
- Лучшая изоляция памяти между процессами

7. **Недостатки MPI**:
   - Более сложная программная модель
   - Значительные накладные расходы на коммуникацию для малых задач
   - Требуется явный обмен граничными данными
8. **Общий вывод**: MPI является предпочтительным выбором для распределённых вычислений на кластере, особенно для больших задач. OpenMP for лучше подходит для общей памяти с малым и средним количеством потоков, а OpenMP task показывает наилучшие результаты при тщательной настройке размера блоков задач.

2 Анализ производительности на одной нити

При сравнении времени выполнения программ на одной вычислительной нити для данных размером 66 и 130, для больших данных эффект практически нивелируется, наблюдается ситуация: параллельные версии, запущенные с параметром `OMP_NUM_THREADS=1`, демонстрируют более высокую производительность, чем чисто последовательная программа. Данный эффект объясняется следующими факторами:

1. **Дополнительные оптимизации в параллельных версиях**: При подготовке к распараллеливанию были внесены улучшения, которые также повысили эффективность последовательного выполнения:
   - Использование указателей на строки матриц для уменьшения вычисления индексов
   - Предвычисление констант (например, `inv_n2 = 1.0/(N*N)`)
   - Замена операции деления на умножение (`*0.125` вместо `/8.0`)
   - Локальные переменные для промежуточных вычислений
2. **Разные алгоритмы вычисления невязки**:
   - В последовательной версии: прямое обновление глобальной переменной `eps` через макрос `Max`
   - В параллельных версиях: использование локальной переменной с последующим однократным присваиванием
3. **Компенсация накладных расходов OpenMP**: Хотя среда OpenMP добавляет некоторые издержки (инициализация, проверка директив), они полностью компенсируются перечисленными выше оптимизациями.

Таким образом, значительное ускорение параллельных версий даже на одной нити объясняется не распараллеливанием, а **совокупностью программных оптимизаций**, которые были внесены для повышения эффективности параллельного выполнения, но также радикально улучшили и последовательную производительность.